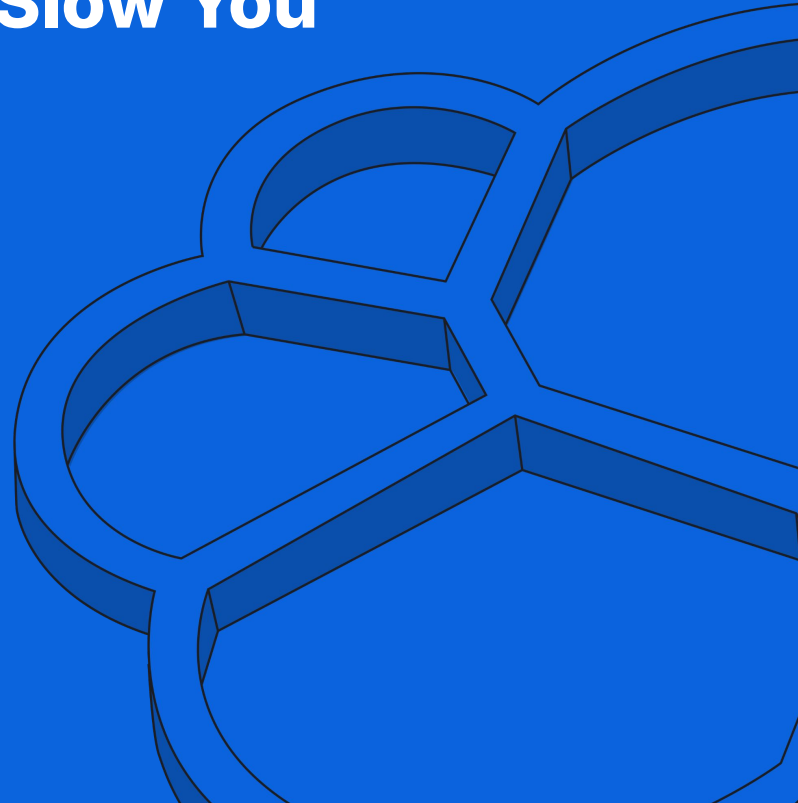


The Performance Paradox: When Slower Code Doesn't Slow You Down

22nd Nov 2025

Anderson Queiroz

Sr Software Engineer, Elastic



Ingest GZIP-compressed logs

"The concatenation of a sequence of gzipped files is equivalent to the gzip of the concatenation of the sequence."

Go's `compress/gzip` docs

Ingest GZIP-compressed logs

in other words, read a GZIP
files, decompress it and
ingest the plain data



On-the-fly decompression: no need for extra storage or loading the whole file into memory



Supports offset tracking / partial reading:
can resume if the file wasn't fully read. Only
one in the market ;)



Identifies files by fingerprint of the
decompressed data - can identify a file 1st
seem as plain, then seem as a GZIP file



Integrity Check, checksum (CRC32) and size
verification, happens at the end and just logs
an error

Filebeat and Elastic Agent

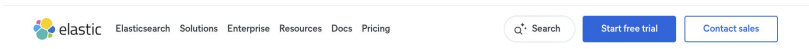
Filebeat

Filebeat is a lightweight shipper for logs

"Whether you're collecting from security devices, cloud, containers, hosts, or OT, Filebeat helps you keep the simple things simple by offering a lightweight way to forward and centralize logs and files."

It's the application which actually does the heavy lifting: reading, processing and shipping the logs to Elasticsearch or to other storages or for further processing.

The Elastic Agent configures and runs a filebeat under the hood. So new features for log collection are, mostly, developed on Filebeat and the agent just needs to configure/enable them.



Filebeat

Lightweight shipper for logs

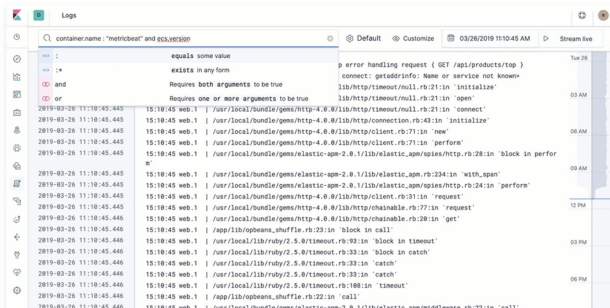
Whether you're collecting from security devices, cloud, containers, hosts, or OT, Filebeat helps you keep the simple things simple by offering a lightweight way to forward and centralize logs and files.

Download

[Filebeat documentation](#) →

Aggregate, "tail -f" & search

After you start Filebeat, open the Logs UI and watch your files being tailed right in Kibana. Use the search bar to filter by service, app, host, datacenter, or other criteria to track down curious behavior across your aggregated logs.



Elastic Agent: all in "one"

"Elastic Agent is a single, unified way to add monitoring for logs, metrics, and other types of data to a host."

The Elastic Agent, or "agent" as we say, is a centrally managed, easy to deploy and configure solution.

Users use [integrations](#) to configure it to observe and protect all sorts of endpoints.



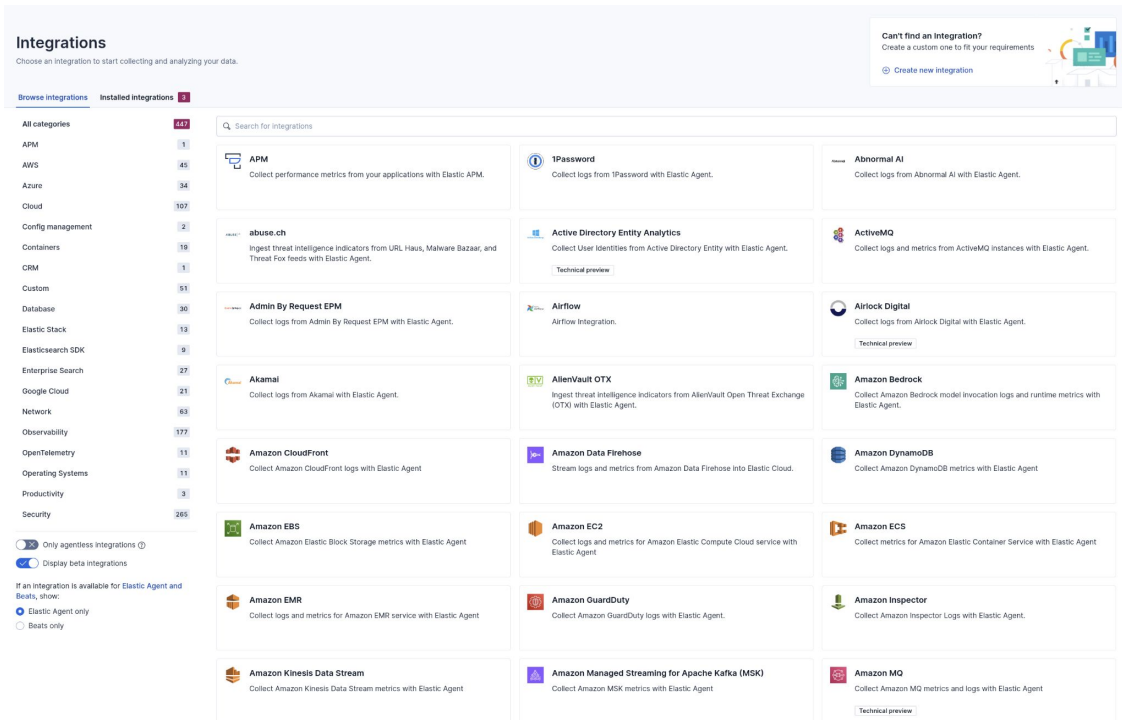
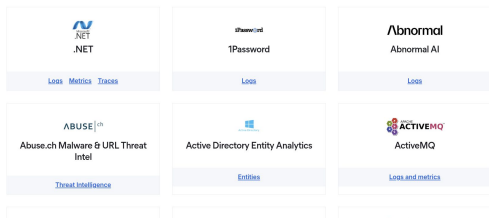
Elastic integrations

Stream in logs, metrics, traces, content, and more from your apps, endpoints, infrastructure, cloud, network, workplace tools, and every other common source in your ecosystem. Send alerts to your notification tool of choice. Connect to all the systems that matter with ease.

Search Integrations

All Solutions Search Security Observability

AWS Azure Cloud Config management Conference Cloud Containers CRM Custom Databases Elastic Agent Enabled Elastic Stack
Entity Analytics Google Cloud Kubernetes Languages Message Queue Monitoring Network Notification Orchestration OS & System
Productivity Security Support Threat Intelligence Ticketing Version Control Vulnerability management Web



Filebeat inputs: it reads data from

- [AWS CloudWatch](#)
- [AWS S3](#)
- [Azure Event Hub](#)
- [Azure Blob Storage](#)
- [Benchmark](#)
- [CEL](#)
- [Cloud Foundry](#)
- [CometD](#)
- [Container](#)
- [Entity Analytics](#)
- [ETW](#)
- [filestream](#)
- [GCP Pub/Sub](#)
- [Google Cloud Storage](#)
- [HTTP Endpoint](#)
- [HTTP JSON](#)
- [journald](#)
- [Kafka](#)
- [Log](#) (deprecated in 7.16.0, use [filestream](#))
- [MQTT](#)
- [NetFlow](#)
- [Office 365 Management Activity API](#)
- [Redis](#)
- [Salesforce](#)
- [Stdin](#)
- [Streaming](#)
- [Syslog](#)
- [TCP](#)
- [UDP](#)
- [Unified Logs](#)
- [Unix](#)
- [winlog](#)

Filebeat outputs: it sends data to

- [Elastic Cloud Hosted](#)
- [Elasticsearch](#)
- [Logstash](#)
- [Kafka](#)
- [Redis](#)
- [File](#)
- [Console](#)
- [Discard](#)

The screenshot shows the Elastic Docs website. At the top, there's a navigation bar with the Elastic logo and links for Products, Solutions, Why Elastic?, Resources, Pricing, and Docs. A search bar and buttons for 'Start free trial' and 'Contact sales' are also present. Below the navigation bar, the page title 'Docs' is displayed. On the left, a sidebar menu shows a tree structure of documentation topics, with 'Output' expanded and 'Elastic Cloud Hosted' selected. The main content area is titled 'Configure the output' and includes a 'Stack' button. The text explains that Filebeat outputs are configured in the 'filebeat.yml' file. A list of supported outputs is provided, including Elastic Cloud Hosted, Elasticsearch, Logstash, Kafka, Redis, File, Console, and Discard. At the bottom, navigation links for 'Previous: Live reloading' and 'Next: Elastic Cloud Hosted' are shown.

elastic Products Solutions Why Elastic? Resources Pricing Docs

Q Search Start free trial Contact sales

Docs Release notes Troubleshoot Reference

Reference

Elastic Docs / Reference / Ingestion tools / Beats / Filebeat / Configure

Current version (9.0+) ▾

Configure the output

Stack

You configure Filebeat to write to a specific output by setting options in the Outputs section of the `filebeat.yml` config file. Only a single output may be defined.

The following topics describe how to configure each supported output. If you've secured the Elastic Stack, also read [Secure](#) for more about security-related configuration options.

- [Elastic Cloud Hosted](#)
- [Elasticsearch](#)
- [Logstash](#)
- [Kafka](#)
- [Redis](#)
- [File](#)
- [Console](#)
- [Discard](#)

View as Markdown Report an issue Edit this page Learn how to contribute

Previous < Live reloading

Next > Elastic Cloud Hosted

Development process

Reading GZIP files

Development workflow

```
filebeat.inputs:  
- type: filestream  
  id: "test-filestream"  
  paths:  
    - /var/some-app/app.log*  
  gzip_experimental: true
```



RFC + PoC: ensure it's viable



Development, testing and review



Merge → Further tests on CI



Benchmark and final review

Benchmarks

We have two different types of benchmark

End-to-end

We have an internal tool called "benchbuilder" to perform end-to-end tests of the beats (filebeat, metricbeat, and so on) and the Elastic Agent.

It focuses on user experience, using events per second (EPS) as its key metric

More complex and requires extensive setup

Go / micro-benchmarks

Regular Go benchmarks, testing a function or small component to access a new implementation is indeed better or we aren't degrading performance with new features or refactors

Easy and cheap to develop and run

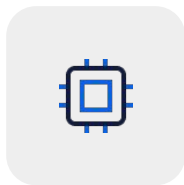
What we want to measure?

- many small files 1000s of 10MB files (Kubernetes log rotation)
- 1 huge file, several GBs (ensure it can work with files larger than the memory available)



Is it slower, the same
or faster?

EPS:
Events per second



How much does this
increase the system
load:

CPU usage



What's the new
memory footprint?

Memory usage

Why to measure?

Common sense: A Clear Trade-Of



Decompression = CPU Work



More CPU Work = Slower Performance



Slower reading $\Rightarrow$ Events Per Second (EPS) will be reduced

PoC benchmarks

Before starting developing the feature we wanted to make sure it was viable

```
› benchstat plain.mac.out gzip.mac.out
```

```
goos: darwin
```

```
goarch: arm64
```

```
pkg: github.com/elastic/beats/v7/filebeat/input/filestream
```

```
cpu: Apple M1 Max
```

	plain.mac.out		gzip.mac.out
	sec/op	sec/op	vs base
Gzip/static-line/1.0_MB-10	12.27m ± 1%	14.78m ± 0%	+20.54% (p=0.000 n=10)
Gzip/static-line/5.2_MB-10	60.05m ± 1%	72.95m ± 0%	+21.48% (p=0.000 n=10)
Gzip/static-line/26_MB-10	300.0m ± 0%	362.5m ± 0%	+20.82% (p=0.000 n=10)
Gzip/static-line/52_MB-10	598.0m ± 0%	723.1m ± 0%	+20.92% (p=0.000 n=10)
Gzip/static-line/105_MB-10	1.199 ± 0%	1.450 ± 2%	+20.90% (p=0.000 n=10)
Gzip/static-line/262_MB-10	3.010 ± 0%	3.616 ± 1%	+20.15% (p=0.000 n=10)
Gzip/static-line/524_MB-10	6.162 ± 0%	7.236 ± 0%	+17.42% (p=0.000 n=10)
Gzip/static-line/1.0_GB-10	12.05 ± 0%	14.53 ± 0%	+20.53% (p=0.000 n=10)
Gzip/random-line/1.0_MB-10	4.890m ± 0%	13.017m ± 0%	+166.18% (p=0.000 n=10)
Gzip/random-line/5.2_MB-10	23.86m ± 0%	63.93m ± 0%	+167.94% (p=0.000 n=10)
Gzip/random-line/26_MB-10	119.7m ± 0%	321.3m ± 0%	+168.47% (p=0.000 n=10)
Gzip/random-line/52_MB-10	240.5m ± 0%	643.9m ± 0%	+167.69% (p=0.000 n=10)
Gzip/random-line/105_MB-10	481.5m ± 0%	1287.3m ± 0%	+167.38% (p=0.000 n=10)
Gzip/random-line/262_MB-10	1.204 ± 0%	3.220 ± 0%	+167.50% (p=0.000 n=10)
Gzip/random-line/524_MB-10	2.411 ± 0%	6.423 ± 0%	+166.46% (p=0.000 n=10)
Gzip/random-line/1.0_GB-10	4.828 ± 0%	13.047 ± 2%	+170.21% (p=0.000 n=10)
geomean	415.9m	746.6m	+79.49%

Plain vs GZIP



Reading GZIP files

Development workflow

```
filebeat.inputs:  
- type: filestream  
  id: "test-filestream"  
  paths:  
    - /var/some-app/app.log*  
gzip_experimental: true
```



~~RFC + PoC: ensure it's viable~~



~~Development, testing and review~~



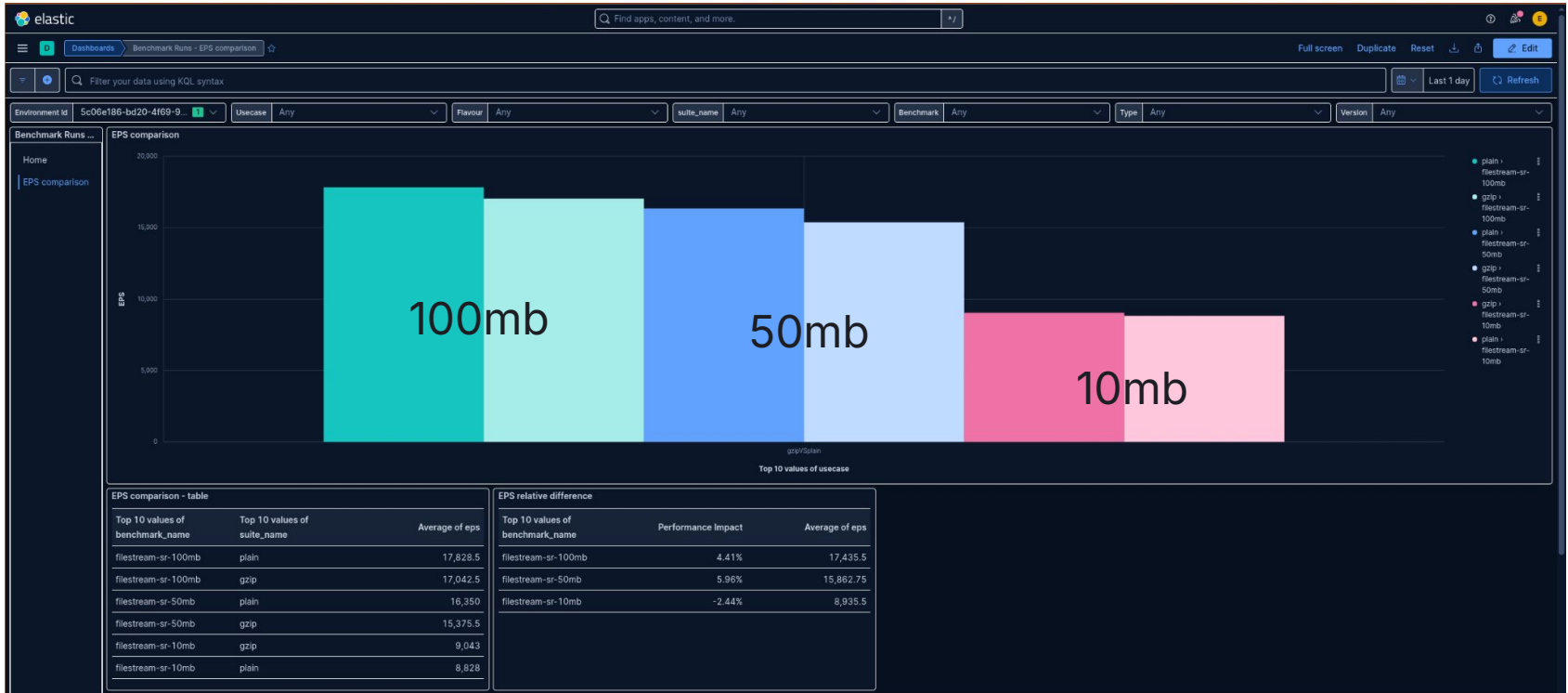
~~Merge → Further tests on CI~~



Benchmark and final review

Benchmarks

1st EPS comparison



Small difference between EPS output

Odd, GZIP was faster. WHY?!

This looks off to me, is there a reasonable explanation?

Screenshot 2025-08-04 at 18.04.43.png ▼

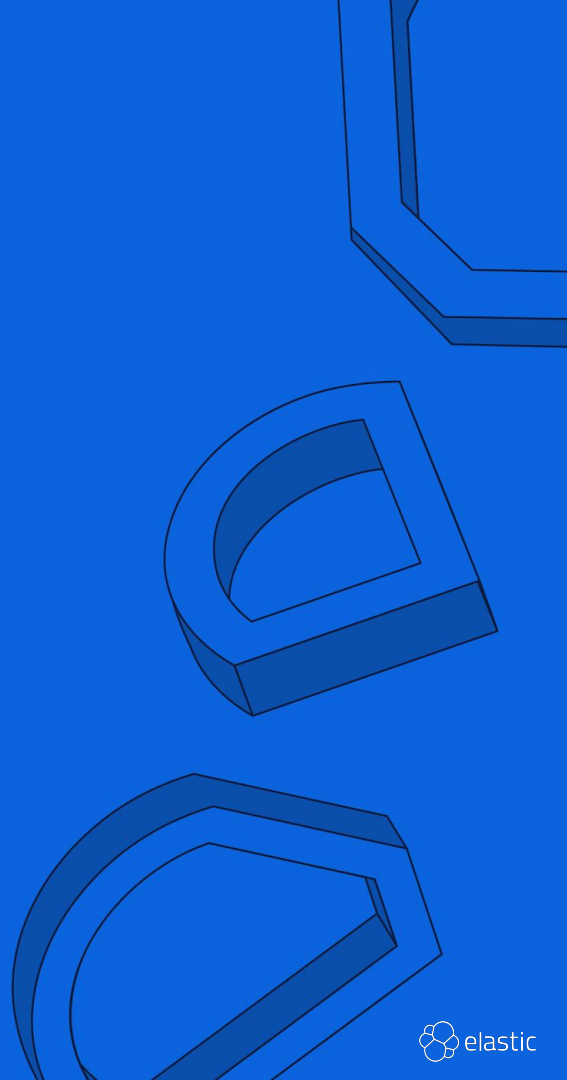
r-10mb	gzip
r-10mb	plain

It cannot possible be that gzip is faster than plain text.

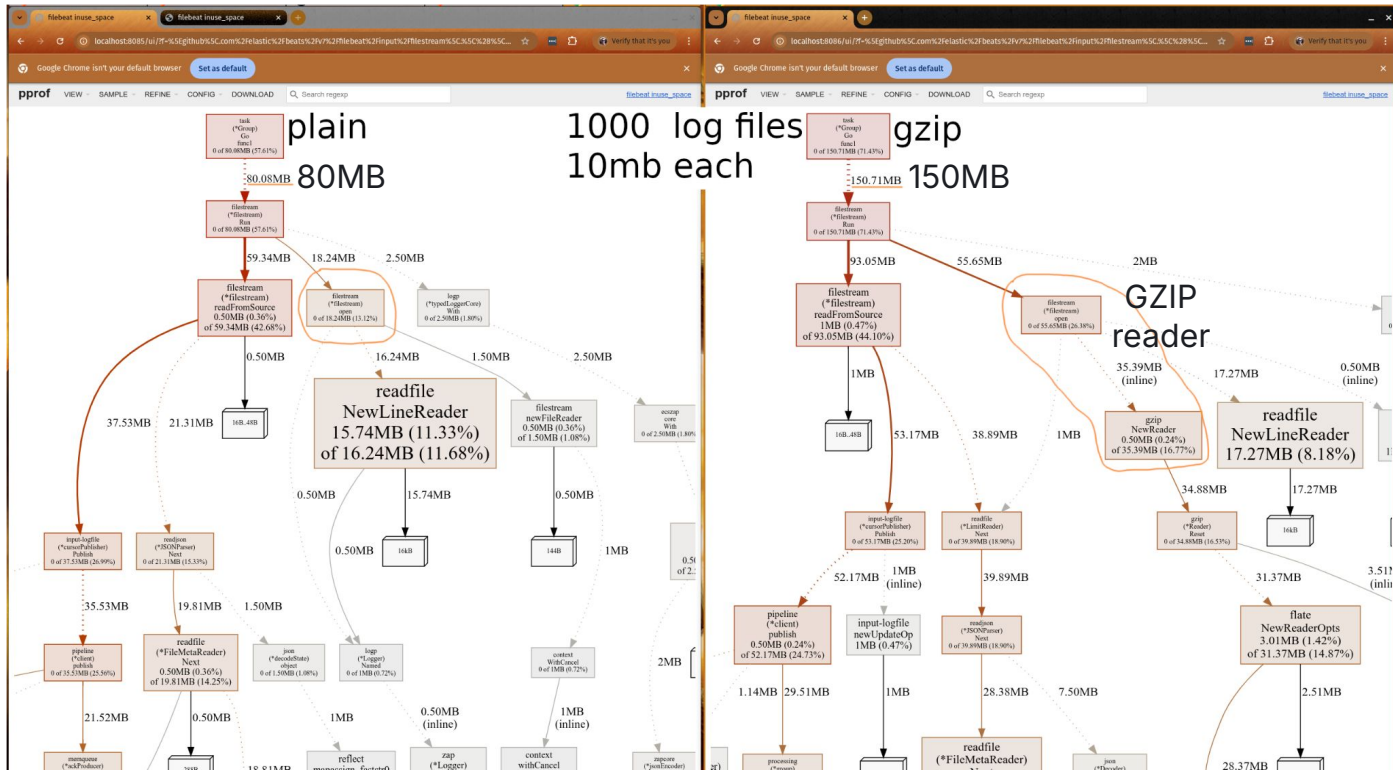
filestream-sr-10mb	gzip	9,043
filestream-sr-10mb	plain	8,828

Memory analysis:

Go benchmark

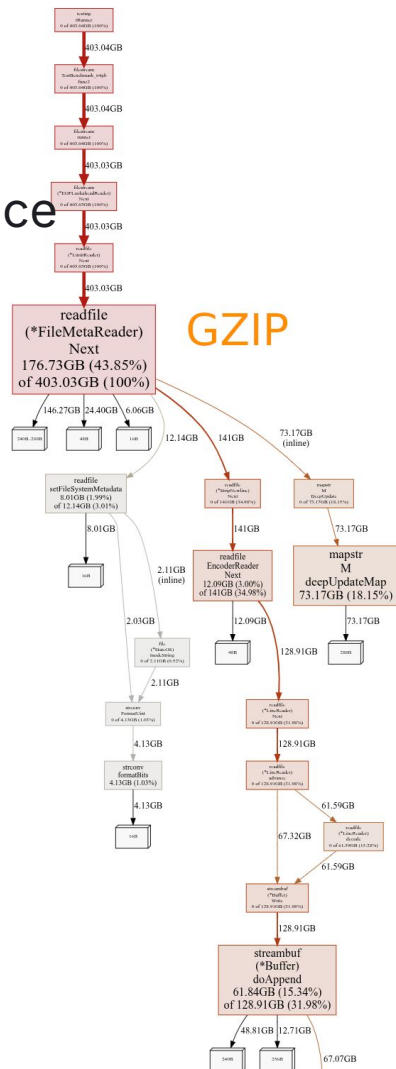
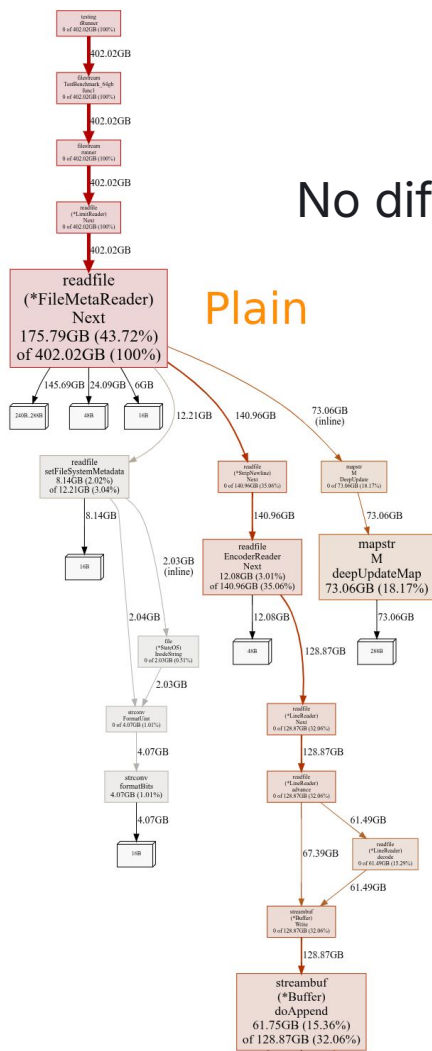


Memory usage: 1000×10MB file



Memory usage: 1×64GB file

No difference



Memory usage: conclusion

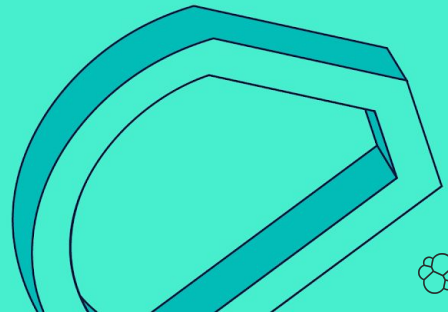
- 100Kb increase in memory usage per file.

Mitigation:

- **limiting the number of files read concurrently.** On Filebeat terms, limit the number of *harvesters*.
- **closing files as soon as then end is reached.** It's the behaviour for GZIP files as no data should be appended to them.

CPU usage:

Go benchmark



CPU usage

Just as for the memory benchmarks:

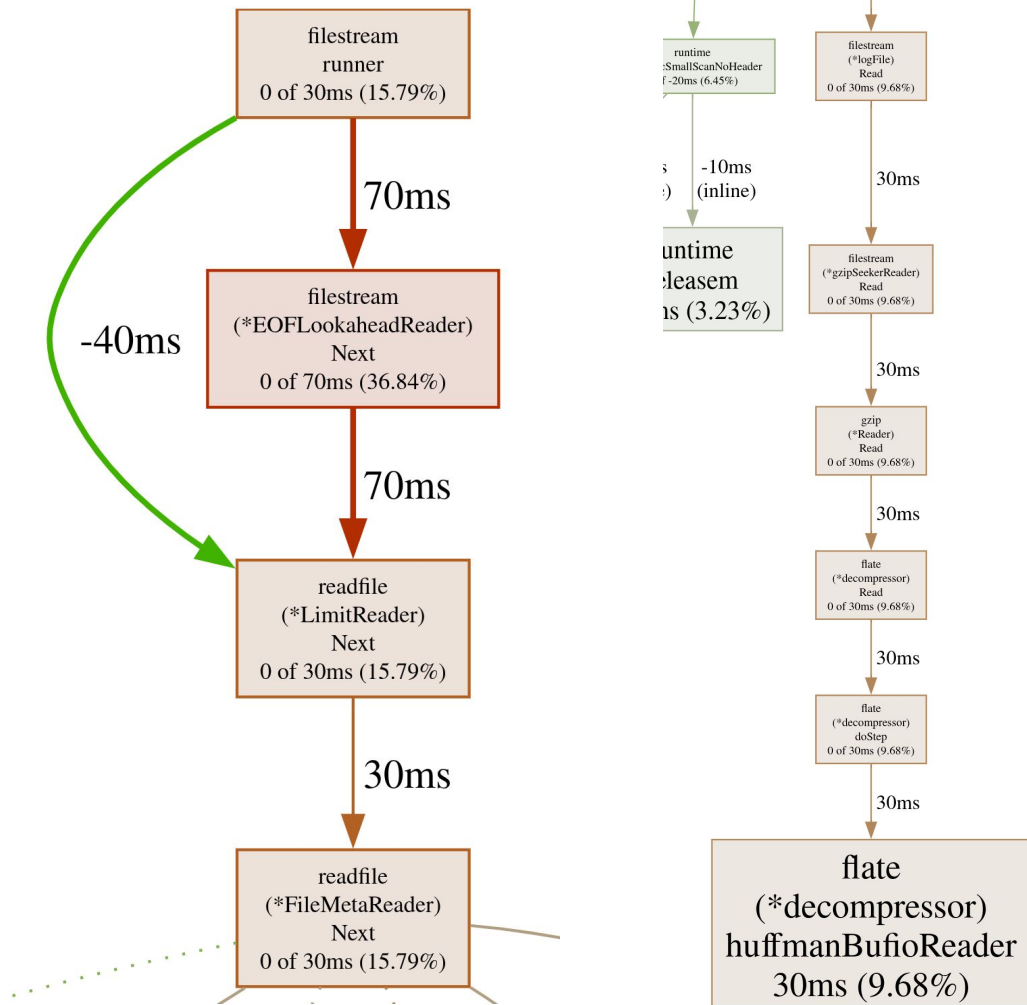
A local benchmark was conducted to evaluate the file reading process in isolation.

This test instantiated the reader abstraction used by filestream, excluding overhead from other Filebeat components like processors and outputs.

We see a increase in CPU usage if we look only at reading the files.

```
go test -v -run TestBenchmark_64gb -timeout=0
=== RUN   TestBenchmark_64gb
=== RUN   TestBenchmark_64gb/plain
=== RUN   TestBenchmark_64gb/gzip
--- PASS: TestBenchmark_64gb (1130.28s)
    --- PASS: TestBenchmark_64gb/plain (251.57s)
    --- PASS: TestBenchmark_64gb/gzip (329.39s)
PASS
```

CPU profile



CPU profile

Plain



GZIP

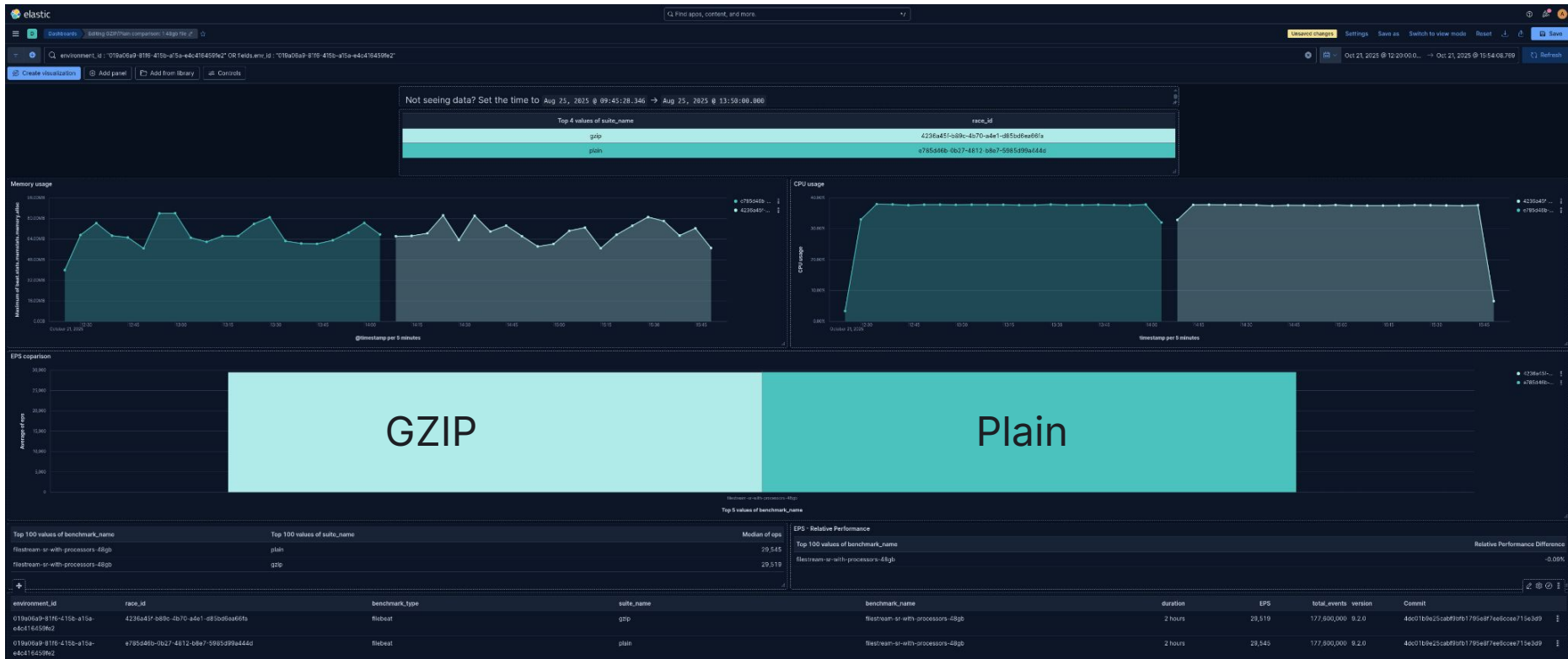


CPU usage: conclusion

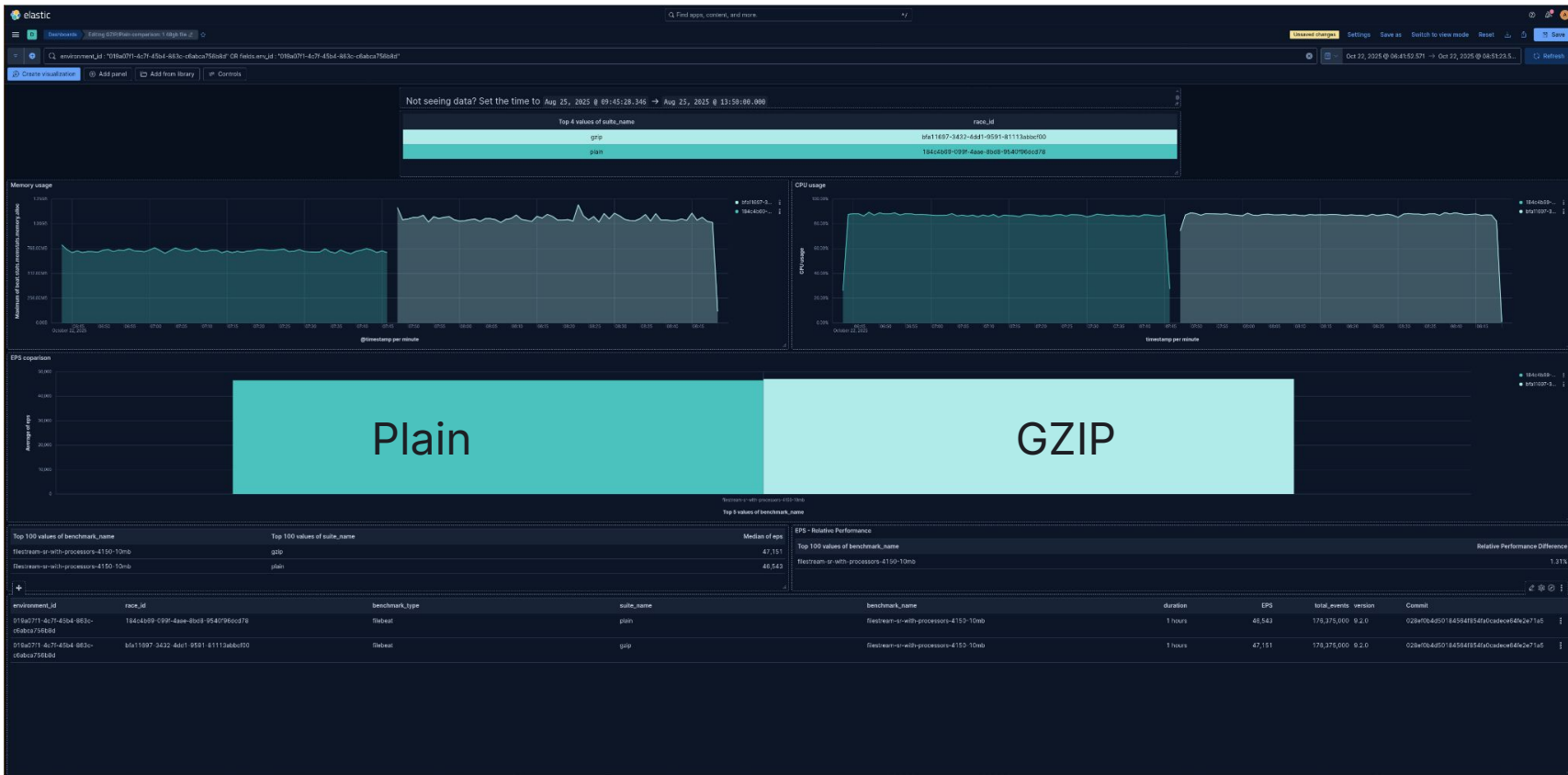
- Increase due to the *EOFLookaheadReader* - signals EOF for GZIP files
- Decompression also increases CPU usage what a surprise!
- Decompression isn't really expensive

Final end-to-end benchmarks

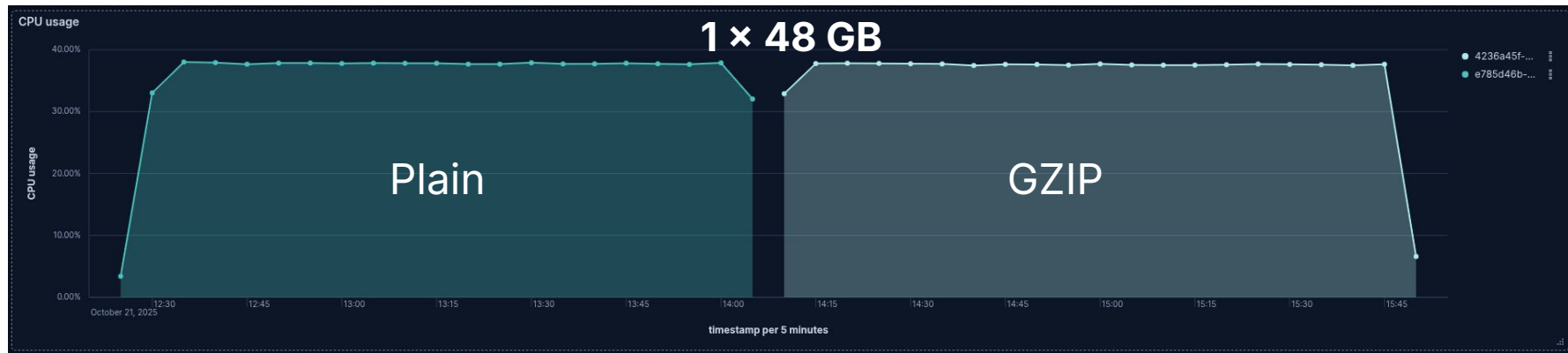
1 × 48GB file



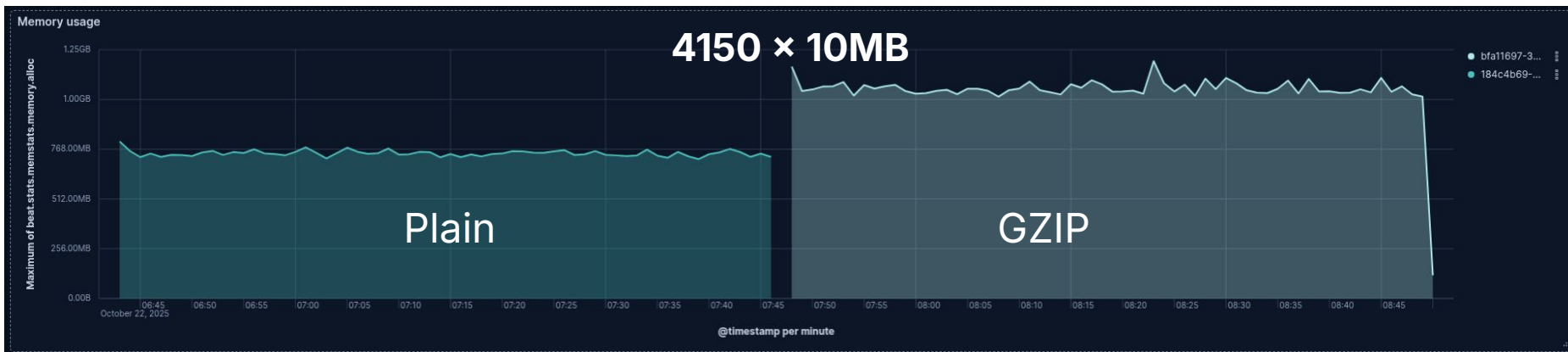
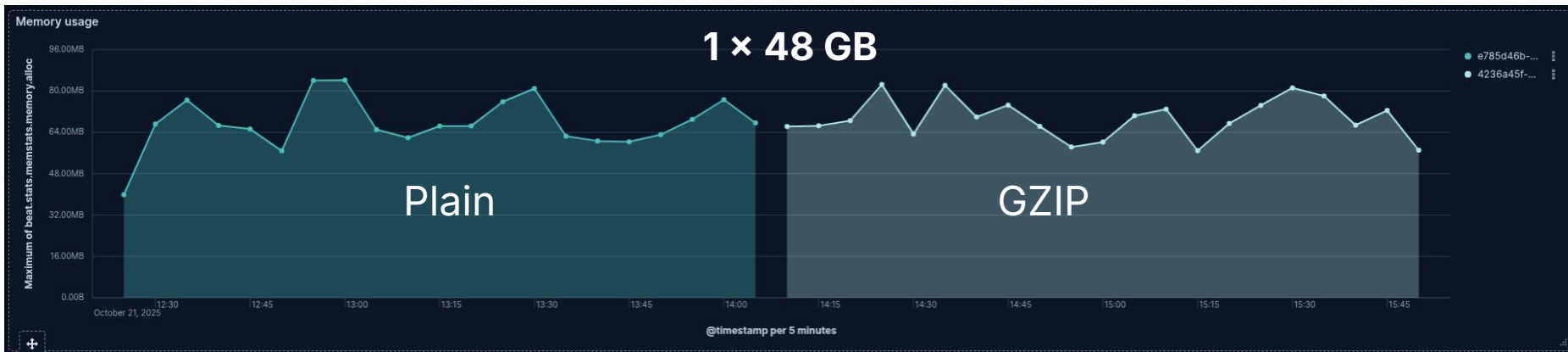
4150 × 10MB files



CPU usage



Memory usage



EPS comparison

1 × 48 GB



4150 × 10MB



EPS comparison

1 × 48 GB

EPS - Relative Performance

Top 100 values of benchmark_name

Relative Performance Difference

filestream-sr-with-processors-48gb

-0.09%

environment_id	race_id	benchmark_type	suite_name	benchmark_name	duration	EPS	total_events	version	Commit
D15a0a0-819e-415b-a15a-e1ca184f39a2	423ea4f1-bd8e-4b70-a4e1-e8f5a0ea66fa	filebeat	grp	filestream-sr-with-processors-48gb	2 hours	29,519	177,605,000	9.2.0	4dc01b5e25cab0f0b1795e8f7ee0cae715e3d9
D15a0a0-819e-415b-a15a-e1ca184f39a2	e785d6db-0a27-4812-bb67-5985d93a444d	filebeat	plain	filestream-sr-with-processors-48gb	2 hours	29,545	177,605,000	9.2.0	4dc01b5e25cab0f0b1795e8f7ee0cae715e3d9

4150 × 10MB

EPS - Relative Performance

Top 100 values of benchmark_name

Relative Performance Difference

filestream-sr-with-processors-4150-10mb

1.31%

environment_id	race_id	benchmark_type	suite_name	benchmark_name	duration	EPS	total_events	version	Commit
019a0791-4c7f-45b4-863c-cfa0ca756b0d	18dc0b80-099f-4aee-8ba9-05a0966ac76	filebeat	plain	filestream-sr-with-processors-4150-10mb	1 hours	48,543	176,375,000	9.2.0	028e49b4dc501845e4f854f0a0ca0e646e2a71a5
019a0791-4c7f-45b4-863c-cfa0ca756b0d	0fa11697-3432-4a61-9591-81113abb00	filebeat	grp	filestream-sr-with-processors-4150-10mb	1 hours	47,151	176,375,000	9.2.0	028e49b4dc501845e4f854f0a0ca0e646e2a71a5

Understanding Filebeat

In a nutshell: How Filebeat works?

Filebeat, reads the data (the logs), process it, then ships it away

Input: filestream

Monitor the file system, find log files, read the files, keep track of current offset for each file, transform each log into filebeat's internal model

Pipeline / Queue

Apply configurable processors which can do all sorts of transformations such as add, remove fields, JSON parsing and field expansion and so on.

Output

Send out the events (the logs). Usually Elasticsearch, but also Logstash, Kafka, Redis and so on.

In a nutshell: How filestream works?

How logs are ingested



File watcher

The file watcher scans the file system every 10s (configurable) to detect:

- new files
- data appended to a file
- a file is truncated
- a file is deleted

For each, an event is sent



Prospector

For each event, it decides if a harvester should:

- start
- stop
- continue
- restart (reset the offset to 0 and start from the beginning)



Harvester

Actually reads the files.

Reads one line at a time and can produce a single event (log) from multiple lines

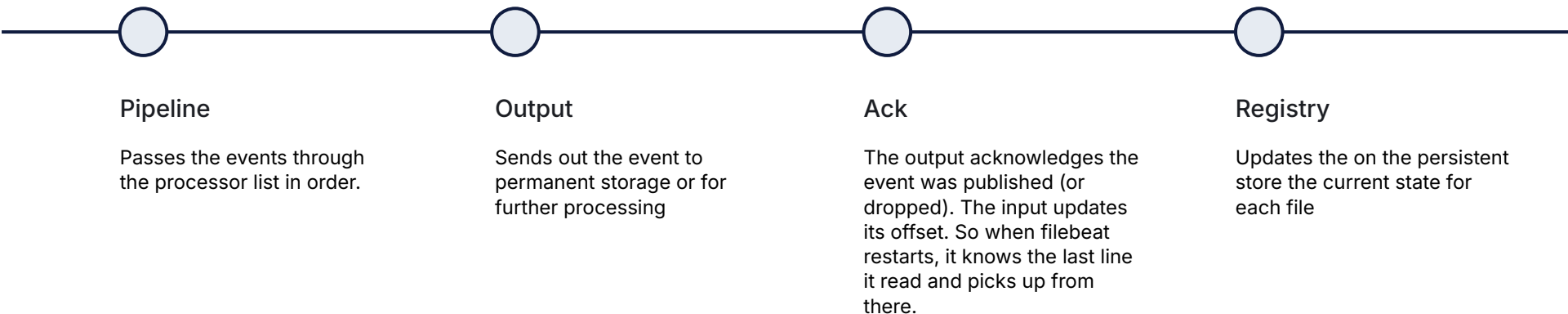


Publisher

Sends the event to the Pipeline for further processing

In a nutshell: How filestream works?

How logs are ingested



Output: shipping data over the network

Sending data over the network adds overhead for sure. Also the Elastic Agent and beats support different output which each one will have its own performance and limitations.

We have a discard output, which, immediately discards any data it receives, allowing us to minimise the impact of sending data and develop without any extra setup.

Benchbuilder does not supports it, but local tests showed it did not affect the performance of ingesting *plain* or *gzip* files.

The screenshot shows the Elastic Docs website. The main heading is 'Configure the Elasticsearch output'. Below it, there's a section 'Example configuration:' with a code block showing a basic configuration for the Elasticsearch output. To the right of the code block, there's a list of authentication methods: Basic authentication credentials (username and password), Token-based (API key) authentication, and Public Key Infrastructure (PKI) certificates. Below this, there are three more code blocks showing configurations for Basic authentication, API key authentication, and PKI certificate authentication. On the right side of the page, there's a sidebar with a list of 'On this page' links, including Compatibility, Configuration options, enabled, hosts, compression_level, escape_html, worker or workers, loadbalance, api_key, username, password, parameters, protocol, path, headers, proxy_disable, proxy_url, proxy_headers, Index, Indices, ilm, pipeline, pipelines, max_retries, bulk_max_size, backoff.init, backoff.max, idle_connection_timeout, timeout, allow_older_versions, ssl, kerberos, queue, non_indexable_policy, preset, and Elasticsearch APIs. The Elastic logo is in the bottom right corner.

Elastic Docs / Reference / Ingestion tools / Beats / Filebeat / Configure / Output

Configure the Elasticsearch output

Stack

The Elasticsearch output sends events directly to Elasticsearch using the Elasticsearch HTTP API.

Example configuration:

```
output.elasticsearch:
  hosts: ["https://myEShost:9200"]
```

To enable SSL, add `https` to all URLs defined under `hosts`.

When sending data to a secured cluster through the `elasticsearch` output, Filebeat can use any of the following authentication methods:

- Basic authentication credentials (username and password).
- Token-based (API key) authentication.
- Public Key Infrastructure (PKI) certificates.

Basic authentication:

```
output.elasticsearch:
  hosts: ["https://myEShost:9200"]
  username: "filebeat_writer"
  password: "YOUR_PASSWORD"
```

API key authentication:

```
output.elasticsearch:
  hosts: ["https://myEShost:9200"]
  api_key: "ZGVVYm9uX011SFRhZQ:KxRyE41RrSowBkQhMeA"
```

PKI certificate authentication:

```
output.elasticsearch:
  hosts: ["https://myEShost:9200"]
  ssl.certificate: "/etc/pki/client/cert.pem"
  ssl.key: "/etc/pki/client/cert.key"
```

See [Secure communication with Elasticsearch](#) for details on each authentication method.

Compatibility

Current version (9.0+)

- View as Markdown
- Report an issue
- Edit this page
- Learn how to contribute

On this page

- Compatibility
- Configuration options
- enabled
- hosts
- compression_level
- escape_html
- worker or workers
- loadbalance
- api_key
- username
- password
- parameters
- protocol
- path
- headers
- proxy_disable
- proxy_url
- proxy_headers
- Index
- Indices
- ilm
- pipeline
- pipelines
- max_retries
- bulk_max_size
- backoff.init
- backoff.max
- idle_connection_timeout
- timeout
- allow_older_versions
- ssl
- kerberos
- queue
- non_indexable_policy
- preset
- Elasticsearch APIs

Processors: impact on overall performance

Logs, or events as we call them, go one by one from the input to the pipeline. Passing by each processor in sequence.

One guarantee we give is if a processor fails, we can still deliver the original event this processor received. All further processors are ignored.

Processors which perform destructive actions must work on a copy of the event, allowing to “rollback” on error.

The screenshot shows the Elastic Docs interface for the topic "Filter and enhance data with processors". The page is part of the "Elastic Docs" collection, specifically under "Reference" > "Ingestion tools" > "Beats" > "Filebeat" > "Configure". The current version is 8.0+.

Filter and enhance data with processors

Your use case might require only a subset of the data exported by Filebeat, or you might need to enhance the exported data (for example, by adding metadata). Filebeat provides a couple of options for filtering and enhancing exported data.

You can configure each input to include or exclude specific lines or files. This allows you to specify different filtering criteria for each input. To do this, you use the `include_lines`, `exclude_lines`, and `exclude_files` options under the `filebeat.inputs` section of the config file (see [inputs](#)). The disadvantage of this approach is that you need to implement a configuration option for each filtering criteria that you need.

Another approach (the one described here) is to define processors to configure global processing across all data exported by Filebeat.

Processors

You can [define processors](#) in your configuration to process events before they are sent to the configured output. The libbeat library provides processors for:

- reducing the number of exported fields
- enhancing events with additional metadata
- performing additional processing and decoding

Each processor receives an event, applies a defined action to the event, and returns the event. If you define a list of processors, they are executed in the order they are defined in the Filebeat configuration file.

```
event -> processor 1 -> event1 -> processor 2 -> event2 ...
```

Important

It's recommended to do all drop and renaming of existing fields as the last step in a processor configuration. This is because dropping or renaming fields can remove data necessary for the next processor in the chain, for example dropping the `source.ip` field would remove one of the fields necessary for the `community_id` processor to function. If it's necessary to remove, rename or overwrite an existing event field, please make sure it's done by a corresponding processor (`drop_fields`, `rename` or `add_fields`) placed at the end of the processor list defined in the input configuration.

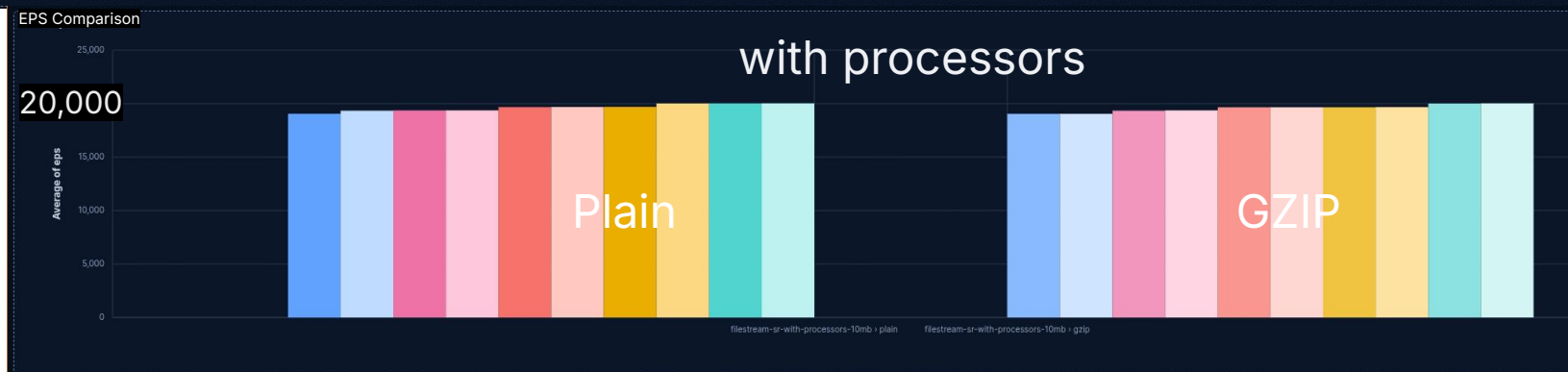
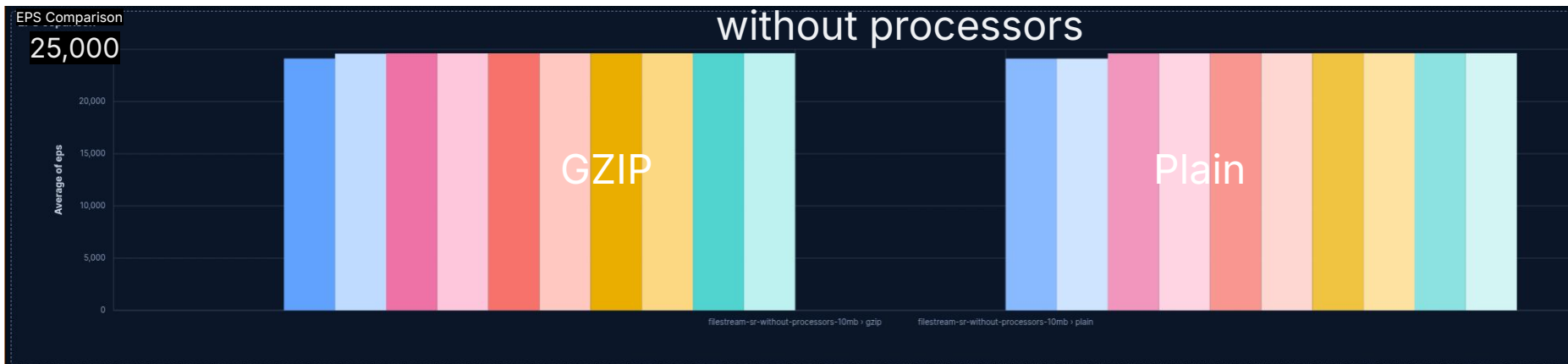
Drop event example

The following configuration drops all the DEBUG messages.

```
processors:
  - drop_event:
```

Remove the processors

EPS comparison



Remove the processors

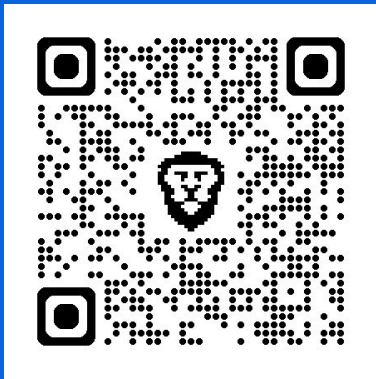
Top 100 values of benchmark_name	Top 100 values of suite_name	Median of eps
filestream-sr-without-processors-100mb	plain	35,448
filestream-sr-without-processors-100mb	gzip	35,240
filestream-sr-without-processors-10mb	plain	24,618.5
filestream-sr-without-processors-10mb	gzip	24,615.5
filestream-sr-with-processors-10mb	plain	19,679
filestream-sr-with-processors-10mb	gzip	19,657.5
filestream-sr-with-processors-100mb	plain	19,438
filestream-sr-with-processors-100mb	gzip	19,414

EPS - Relative Performance	
Top 100 values of benchmark_name	Relative Performance Difference
filestream-sr-with-processors-100mb	0.03%
filestream-sr-with-processors-10mb	-0.39%
filestream-sr-without-processors-100mb	-0.43%
filestream-sr-without-processors-10mb	0.19%

Conclusion

DO NOT GUESS, MEASURE!

- Local gain or losses might not impact the whole
- Reading GZIP files is slower than reading plain files
- However, it does not affect Filebeat/Elastic Agent performance
- The bottleneck is the queue/pipeline
- There is a ~100KB memory increase per file read
- No CPU increase in the overall performance



github.com/AndersonQ/talks
Email: **me@andersonq.me**

Questions?

Thank you!



LinkedIn: **Anderson Queiroz**
GitHub: **@AndersonQ**



The Search
AI Company